Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

Scan ID 22	Date August 28, 2026
Organization RigoBlock	Repository v3-contracts
Branch feat/hyperliquid-inflight-mitigation	Commit Hash c9bb0495...f75eab2e

Contracts in scope

contracts/protocol/extensions/adapters/AHyperliquid.sol

contracts/protocol/libraries/HyperliquidLib.sol

Code Statistics

Findings 9	Contracts Scanned 2	Lines of Code 253

Findings Summary

Total Findings

- High Risk (0)
- Medium Risk (2)
- Low Risk (6)
- Info (1)
- Best Practices (0)

Code Summary

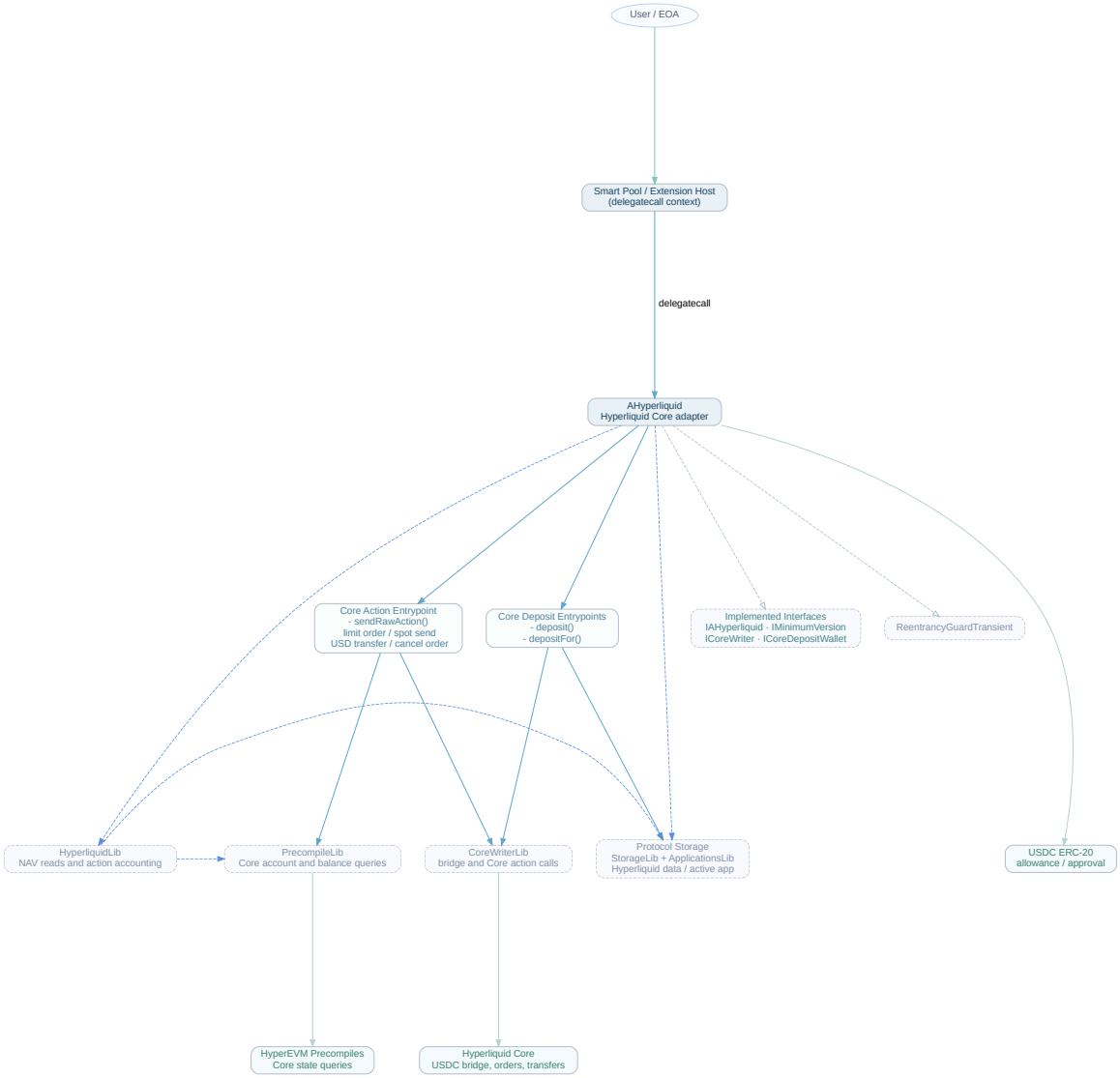
This protocol provides an integration adapter and accounting library for Rigoblock smart pools operating on the HyperEVM network, enabling direct interaction with Hyperliquid Core. Through delegate calls from smart pools, the adapter acts as an execution module that facilitates trading perpetual contracts, managing collateral, and coordinating cross-layer asset transfers between HyperEVM and Hyperliquid Core.

The adapter facilitates bridging USDC into Hyperliquid's perpetual exchange, submitting and canceling limit orders on supported perpetual assets, transferring USD balances between perpetual and spot account classes, and executing spot sends to bridge assets back to the EVM. Complementing the adapter, the accounting library tracks the pool's net asset value (NAV) by querying Hyperliquid precompiles for account margin summaries and spot USDC balances. To ensure accurate NAV pricing and prevent arbitrage during in-flight actions or state synchronization delays, the system records pending actions across composite block identifiers (combining L1 and EVM block numbers) and enforces a temporary NAV lock settlement window.

Main Entry Points

- `deposit(uint256 amount, uint32 destinationDex)`: Invoked by smart pools and pool managers to bridge USDC collateral from HyperEVM into Hyperliquid Core for perpetual trading.
- `depositFor(address recipient, uint256 amount, uint32 destinationDex)`: Invoked by smart pools and pool managers to bridge USDC into Hyperliquid Core designated specifically for the calling pool contract.
- `sendRawAction(bytes calldata data)`: Invoked by smart pools and pool managers to dispatch encoded actions to Hyperliquid Core, including placing limit orders, canceling orders by order ID or client order ID, transferring USD from perp to spot accounts, and executing spot sends to initiate withdrawals.

Code Diagram



1 of 9 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

contracts/protocol/libraries/HyperliquidLib.sol

Limit-order fills can change HyperCore value without arming the NAV settlement lock

Medium Risk

`sendRawAction` permits Core perp limit orders, but the limit-order path neither records an action in `HyperliquidLib` nor updates `lastActionTimestamp`. Consequently, it does not activate the 128-second guard enforced by `getHyperliquidBalances` before returning the HyperCore account value used by NAV accounting.

The relevant order path only forwards the order to HyperCore:

function _placeLimitOrder(bytes calldata params) private {
(uint32 asset, bool isBuy, uint64 limitPx, uint64 sz, bool reduceOnly, uint8 encodedTif, uint128 cloid) =
abi.decode(
params,
(uint32, bool, uint64, uint64, bool, uint8, uint128)
);

```
require(sz > 0, InvalidAmount());
_requireCorePerpAsset(asset);

CoreWriterLib.placeLimitOrder(asset, isBuy, limitPx, sz, reduceOnly, encodedTif, cloid);
```

}

By comparison, only `_bridgeUsdcToCore` and `_spotSend` invoke `HyperliquidLib.recordAction`. The lock itself depends exclusively on the timestamp written through that function:

function _assertNavUnlocked() private view {
uint48 unlockAt = StorageLib.hyperliquidData().lastActionTimestamp;
if (unlockAt != 0) {
require(block.timestamp >= unlockAt + _SETTLEMENT_WINDOW, NavLocked());
}
}

A marketable limit order can fill immediately and alter the Core account value through execution fees, realized PnL, and a newly opened or closed perpetual position before the HyperEVM reader precompile reflects the result. More importantly, a resting limit order can fill later through a Core-side counterparty action, at a time unrelated to the original `sendRawAction` call. Such a later fill has no corresponding EVM adapter invocation and therefore cannot update `lastActionTimestamp` at all.

During the resulting precompile-lag interval, `getHyperliquidBalances` can return a HyperCore balance snapshot that predates the fill instead of reverting with `NavLocked`. NAV-sensitive pool transitions can then price shares using a value that is too high after trading losses or fees, or too low after favorable execution or PnL. This can produce incorrect mint and burn pricing, transferring value between transacting users and existing pool shareholders.

Severity Note:
- getHyperliquidBalances is used as a live input to share mint/burn NAV (NAV accounting itself is not in the

supplied contracts).
- HyperEVM account-margin precompiles can lag Core limit-order fills analogously to the deposit/spot-send lags the 128s lock was written for.
- Fill-induced account-value changes can be material versus pool NAV, not just dust fees.

2 of 9 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

`sendRawAction` fails to activate `Applications.HYPERLIQUID`, omitting HyperCore assets from pool NAV

Medium Risk

The Rigoblock smart pool architecture tracks external non-token holdings by checking active applications stored in `StorageLib.activeApplications()`. During NAV calculation, the valuation extension iterates through registered applications and queries their balances (e.g., via `HyperliquidLib.getHyperliquidBalances`).

In `AHyperliquid.sol`, `StorageLib.activeApplications().storeApplication(uint256(Applications.HYPERLIQUID))` is only executed inside `_bridgeUsdcToCore`. When a pool operates on HyperCore where the account was initialized, funded, or pre-activated directly on Core rather than through `AHyperliquid.deposit`, the manager can execute perpetual trades, cancellations, class transfers, and spot sends via `sendRawAction`. `sendRawAction` requires that `PrecompileLib.coreUserExists(address(this))` is true, but fails to register `Applications.HYPERLIQUID` in `StorageLib.activeApplications()`. Consequently, the pool can accumulate substantial perpetual equity or spot USDC balances on HyperCore while `Applications.HYPERLIQUID` remains unregistered, causing all HyperCore assets and liabilities to be entirely omitted from the pool's NAV calculation.

Severity Note:
- Pool NAV iterates StorageLib.activeApplications() and only then reads HyperCore balances, as described in the finding (not implemented in this file).
- A Core user for the pool address can be created and funded without AHyperliquid.deposit/depositFor.
- Share mint and redeem consume this NAV, so mis-valuation can move value between LPs and traders.

3 of 9 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

Deprecated OpenZeppelin Function

Low Risk

The adapter invokes `usdc.safeApprove(depositWallet, 1)`. This API is deprecated in the OpenZeppelin version targeted by the project, so the call relies on a legacy token-allowance interface and may create compatibility or maintenance issues when the dependency is upgraded.

4 of 9 Findings — contracts/protocol/extensions/adapters/AHyperliquid.sol

Large Numeric Literal — Low Risk

`_ASSET_ID_CORE_SPOT_BASE` is initialized with the numeric literal `10_000`. The value is a power-of-ten scaling value expressed in full decimal form rather than scientific notation, which reduces consistency with the project's representation of large numeric constants and can make the intended magnitude less immediately apparent.

5 of 9 Findings — contracts/protocol/extensions/adapters/AHyperliquid.sol

Literal Instead of Constant — Low Risk

The byte offset literal `4` is used when slicing calldata as `bytes calldata params = data[4:]`. The same statement is reported at three occurrences in `AHyperliquid.sol`. This embeds the ABI function-selector offset directly in each location instead of giving the repeated value a named representation, which makes the relationship between the offset and the calldata layout less explicit and requires each occurrence to be maintained independently.

6 of 9 Findings — contracts/protocol/extensions/adapters/AHyperliquid.sol

Unchecked Return — Low Risk

The return value from `HyperliquidLib.recordAction(amount.toInt256(), false)` is ignored. If the library function returns status or other information about recording the action, the surrounding adapter logic proceeds without observing that result, so a failure or unexpected outcome represented by the return value cannot affect subsequent execution.

7 of 9 Findings — contracts/protocol/extensions/adapters/AHyperliquid.sol

Unused Import — Low Risk

`AHyperliquid.sol` contains the import `ICoreDepositWallet` from `hyper-evm-lib/interfaces/ICoreDepositWallet.sol`, which is reported as unused. The imported symbol does not contribute to the compiled logic at the reported locations, leaving an unnecessary source dependency and obscuring the imports that are required by the adapter.

8 of 9 Findings

contracts/protocol/libraries/HyperliquidLib.sol

Internal Function Used Only Once

Low Risk

`getHyperliquidBalancesUnsafe(address account)` is declared as an internal view helper and is reported as having only one call site. Keeping the logic behind a single-use internal function adds an additional dispatch layer without providing reuse across multiple callers, and the function's implementation is therefore isolated from the only operation that consumes its result.

9 of 9 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

contracts/protocol/libraries/HyperliquidLib.sol

Settlement safety relies on an unenforced 128-second Core-state convergence assumption • Info

The settlement guard releases solely because 128 seconds have elapsed since a recorded deposit or `SPOT_SEND`; it does not verify that the corresponding HyperCore account state, Core spot balance, and EVM-side bridge settlement have actually converged in the HyperEVM precompile views.

`recordAction` stores only a local timestamp and a same-composite-block accounting adjustment:

function recordAction(int256 amount, bool isSpotSend) internal returns (uint64 pendingBefore) {
HyperliquidData storage data = StorageLib.hyperliquidData();
uint256 compositeBlock = _compositeBlockNumber();
if (data.lastActionCompositeBlock != compositeBlock) {
data.inFlightAmount = 0;
data.pendingSpotSend = 0;
data.lastActionCompositeBlock = compositeBlock;
}
data.lastActionTimestamp = uint48(block.timestamp);

```
if (isSpotSend) {
    pendingBefore = data.pendingSpotSend;
    data.pendingSpotSend = pendingBefore + SafeCast.toUint64(uint256(amount));
} else {
    data.inFlightAmount += amount.toInt128();
}
```

}

The safe reader subsequently unlocks based only on elapsed local time:

function _assertNavUnlocked() private view {
uint48 unlockAt = StorageLib.hyperliquidData().lastActionTimestamp;
if (unlockAt != 0) {
require(block.timestamp >= unlockAt + _SETTLEMENT_WINDOW, NavLocked());
}
}

There is no association between a recorded action and a Core settlement identifier, observed Core block, precompile update marker, or bridge-receipt confirmation. In addition, the temporary deposit adjustment is applied only while `lastActionCompositeBlock` equals the current composite block. Once that block changes, `getHyperliquidBalancesUnsafe` stops adding `inFlightAmount`, even if the Core-side reader remains stale; the time lock is the only remaining protection.

If a Core deposit is not visible in the account-margin precompile after the 128-second interval, the pool's EVM USDC has already left the pool while the Core-side value is still absent from the reported application balance. A subsequent NAV update can understate pool value and permit underpriced share issuance. If a `SPOT_SEND` has not converged consistently across the Core spot-balance view and the EVM bridge credit by unlock time, the

reported aggregate value can likewise be temporarily understated or overstated. The contract does not distinguish whether the initiating transaction was included in a small or big HyperEVM block and has no on-chain condition establishing that the external settlement state is current when the lock expires.

Severity Note:
- Hyperliquid Core/precompile views can remain inconsistent with a completed EVM deposit or SPOT_SEND for more than 128 seconds under conditions an attacker can wait on or induce.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.]
No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.